

FileXplorer Pro

IF3001 – Algoritmos y Estructuras de Datos | UCR Sede de Occidente

Simulador de sistema de archivos en memoria RAM, administrado desde una CLI interactiva, implementado en Java con estructuras de datos.

Estructura del proyecto

```
src/main/java/com/mycompany/fileexplorer/
├── Main.java                # Punto de entrada
├── Fileexplorer.java        # Loop principal de la CLI
├── dominio/
│   ├── NodoFS.java         # Clase abstracta base (nodo del árbol)
│   ├── Carpeta.java        # Nodo interno del árbol N-ario
│   ├── Archivo.java        # Nodo hoja del árbol
│   └── SistemaArchivos.java # Gestor del árbol + historial + cola
├── estructuras/
│   ├── ListNode.java       # Nodo genérico enlazado (package-private)
│   ├── LinkedListGeneric.java # Lista enlazada genérica con MergeSort
│   ├── StackGeneric.java   # Pila genérica (LIFO)
│   └── QueueGeneric.java   # Cola genérica (FIFO)
└── utils/
    ├── Command.java        # Parser de comandos de consola
    ├── Reader.java          # Lector de stdin con función de parseo
    └── Writer.java          # Escritor de salida estándar
```

Documento de Diseño y Diagrama de Clases

1. Propósito del modelo

El sistema implementa un **árbol N-ario de sistema de archivos en memoria**. Cada nodo del árbol representa una de dos entidades:

- **Carpeta**: puede contener otros nodos.
- **Archivo**: es un nodo hoja y no contiene hijos.

La estructura de hijos de cada carpeta no se guarda con arreglos ni con listas de Java, sino con una **lista enlazada genérica propia**. Esto permite ver con claridad el uso de referencias y punteros dentro del proyecto.

2. Clases principales del modelo de archivos

2.1 NodoFS (clase abstracta)

Es la clase base de todo nodo del sistema de archivos.

Atributos:

- nombre : String
- fechaCreacion : long
- parent : Carpeta

Rol:

- Define la información común de carpetas y archivos.
 - La referencia `parent` conecta cada nodo con su carpeta contenedora.
 - Esta referencia hace posible reconstruir la ruta completa del nodo.
-

2.2 Carpeta

Representa una carpeta del árbol.

Hereda de: `NodoFS`

Atributos:

- `hijos : LinkedListGeneric<NodoFS>`

Rol:

- Almacena sus hijos en una lista enlazada propia.
- Puede contener tanto `Carpeta` como `Archivo`.
- Cuando se agrega un hijo, primero se actualiza su `parent` y luego se inserta en la lista.

Métodos relevantes:

- `addChild(NodoFS n)`
 - `removeChild(String name)`
 - `findChild(String name)`
 - `listChildren()`
 - `listChildrenLong()`
 - `sortChildBySize()`
-

2.3 Archivo

Representa un archivo del sistema.

Hereda de: `NodoFS`

Atributos:

- `extension : String`
- `tamKB : int`
- `contenido : String`

Rol:

- Es un nodo hoja.
 - No tiene lista de hijos.
 - Solo almacena metadatos y contenido simulado.
-

2.4 SistemaArchivos

Es la clase que administra toda la estructura del árbol.

Atributos:

- `root : Carpeta`
- `current : Carpeta`
- `back : StackGeneric<Carpeta>`
- `forward : StackGeneric<Carpeta>`
- `taskQueue : QueueGeneric<String>`

Rol:

- Mantiene la raíz del árbol.
- Mantiene la carpeta actual.

- Controla historial de navegación con dos pilas.
- Controla tareas en cola con una cola FIFO.

Operaciones principales:

- mkdir
- rmdir
- rm
- touch
- cd
- goToRoot
- ls
- pwd
- back
- forward
- searchDFS
- searchBFS
- queueAdd
- queueRun

3. Estructuras enlazadas usadas por el sistema

3.1 ListNode<T>

Es el nodo básico de la lista enlazada.

Atributos:

- data : T
- next : ListNode<T>

Rol:

- Guarda el dato real.
- Guarda la referencia al siguiente nodo.
- Es la base de la lista, la pila y la cola.

3.2 LinkedListGeneric<T>

Implementa una lista enlazada simple genérica.

Atributos:

- head : ListNode<T>
- size : int

Rol:

- Es la estructura que usan las carpetas para guardar sus hijos.
- Inserta al final recorriendo la lista desde head.
- Permite eliminar, obtener por índice y ordenar con MergeSort.

3.3 StackGeneric<T>

Implementa una pila genérica.

Atributos:

- top : ListNode<T>
- size : int

Rol:

- Usa la misma clase de nodo que la lista enlazada.
- Trabaja en orden LIFO.
- Se usa en el historial de navegación y en la búsqueda DFS.

3.4 QueueGeneric<T>

Implementa una cola genérica.

Atributos:

- head : ListNode<T>
- tail : ListNode<T>
- size : int

Rol:

- Usa nodos enlazados con referencia al siguiente.
- Trabaja en orden FIFO.
- Se usa en la búsqueda BFS y en la cola de tareas.

4. Diagrama conceptual de referencias y punteros

```
NodoFS
  nombre
  fechaCreacion
  parent -> Carpeta

Carpeta extends NodoFS
  hijos -> LinkedListGeneric<NodoFS>

Archivo extends NodoFS
  extension
  tamKB
  contenido

LinkedListGeneric<NodoFS>
  head -> ListNode<NodoFS>
  size

ListNode<NodoFS>
  data -> NodoFS
  next -> ListNode<NodoFS>

SistemaArchivos
  root -> Carpeta
  current -> Carpeta
  back -> StackGeneric<Carpeta>
  forward -> StackGeneric<Carpeta>
  taskQueue -> QueueGeneric<String>
```

Comandos disponibles

Comando	Descripción
ls	Lista el contenido del directorio actual (carpetas primero, luego archivos)
ls -s	Lista el contenido ordenado por tamaño (MergeSort)
mkdir <nombre>	Crea una subcarpeta en el directorio actual
rmdir <nombre>	Elimina una subcarpeta vacía
rm <nombre>	Elimina un archivo
rm -r <nombre>	Elimina una carpeta aunque no esté vacía
touch <nombre>.<ext> <tamKB>	Crea un archivo en el directorio actual
cd <nombre>	Entra a una subcarpeta
cd ..	Sube al directorio padre
cd	Vuelve a la raíz
pwd	Muestra la ruta absoluta del directorio actual
back	Navega hacia atrás en el historial (usa pila back)
forward	Navega hacia adelante en el historial (usa pila forward)
search -dfs <nombre>	Búsqueda en profundidad (DFS iterativo con pila) desde la raíz
search -bfs <nombre>	Búsqueda en anchura (BFS con cola) desde la raíz
queue -add <operacion> <ruta>	Agrega una tarea a la cola de operaciones por lote
queue -run	Procesa todas las tareas en la cola (FIFO)

Resumen de avances

Avance 1 — Estructuras lineales y Árbol N-ario

Implementar y probar de forma aislada las estructuras lineales (Lista, Pila, Cola) y el Árbol N-ario.

Lo que se implementó:

- **LinkedListGeneric<T>** — Lista enlazada genérica con nodos propios (ListNode<T>). Operaciones: add, remove, get, size, isEmpty, sort (MergeSort sobre nodos). Sin uso de java.util.
- **StackGeneric<T>** — Pila genérica sobre ListNode<T>. Operaciones: push, pop, peek, clear, isEmpty, size.
- **QueueGeneric<T>** — Cola genérica con punteros head y tail. Operaciones: enqueue, dequeue, peek, isEmpty, size.

- **NodoFS** — Clase abstracta base con nombre, fechaCreacion y referencia al parent.
- **Carpeta** — Nodo interno del árbol. Mantiene LinkedListGeneric<NodoFS> hijos para almacenar subcarpetas y archivos. Implementa addChild, removeChild, findChild, listChildren, sortChildBySize.
- **Archivo** — Nodo hoja con metadatos: extension, tamKB, contenido (simulado).
- **SistemaArchivos** — Gestor de la raíz del árbol con mkdir, touch, cd, ls básicos.

Avance 2 — Comandos de navegación e historial

Implementar los comandos básicos de navegación (mkdir, cd, ls, etc.) y el historial operativo con dos pilas.

Lo que se implementó:

- **CLI interactiva** — Loop en Fileexplorer.java con prompt estilo [user@arch raiz/\$], parseo de comandos con Command.java.
- **Navegación completa** — mkdir, rmdir, rm, rm -r, touch, ls, ls -s, cd, cd .., cd (volver a raíz), pwd.
- **Historial con pilas** — back y forward usando dos instancias de StackGeneric<Carpeta>. Cada cd empuja el directorio actual en back y limpia forward. back() mueve el directorio actual a forward antes de retroceder.
- **Búsqueda DFS** — Iterativa con StackGeneric<Carpeta>, recorre el árbol en profundidad desde la raíz.
- **Búsqueda BFS** — Con QueueGeneric<Carpeta>, recorre nivel por nivel desde la raíz.
- **Cola de operaciones** — queue -add encola tareas como strings; queue -run las procesa en orden FIFO mostrando simulación de procesamiento.
- **Ordenamiento dinámico** — ls -s invoca sortChildBySize() en Carpeta, que llama a LinkedListGeneric.sort() con MergeSort ($O(n \log n)$).

Avance 3 — Se afinan detalles

Se incorporaron nuevas operaciones al sistema de archivos, mejoras en el recorrido y la búsqueda, manejo de historial de navegación y ordenamiento de listas enlazadas.

CASES AGREGADOS AL MÉTODO DE LA Clase FileExplorer

case "ls"

Este caso obtiene los argumentos del comando para determinar si el listado debe ordenarse por tamaño.

Flujo del método:

1. Se extraen los argumentos recibidos en cmd.
2. Se verifica si el primer argumento es -s.
3. Si -s está presente, se activa la variable orderBySize.
4. Se llama a fs.ls(orderBySize).
5. El resultado se escribe en la salida.

Propósito: mostrar el contenido de la carpeta actual y, opcionalmente, ordenarlo por tamaño.

case "pwd"

Este caso imprime la ruta absoluta de la ubicación actual dentro del árbol de archivos.

Flujo del método:

6. Se invoca fs.pwd().

7. El resultado devuelto se escribe directamente en la salida.

Propósito: permitir al usuario saber exactamente en qué carpeta o archivo se encuentra.

case "back"

Este caso permite regresar a la ubicación anterior dentro del historial de navegación.

Flujo del método:

8. Se llama a `fs.back()`.

9. Si el método retorna `false`, significa que no existe una ubicación anterior.

10. En ese caso, se muestra el mensaje: `back: already at the beginning of history`.

Propósito: navegar hacia atrás en la ruta visitada.

case "forward"

Este caso permite avanzar hacia la siguiente ubicación registrada en el historial.

Flujo del método:

11. Se llama a `fs.forward()`.

12. Si el método retorna `false`, significa que no existe una ubicación posterior.

13. En ese caso, se muestra el mensaje: `forward: already at the end of history`.

Propósito: recuperar una ubicación a la que el usuario ya había accedido antes de usar `back()`.

2. MÉTODOS AGREGADOS EN Clase FileSystem

createFileSystemTree()

Este método construye una estructura de prueba con varios niveles de carpetas y archivos. Su propósito es crear un árbol suficientemente grande para probar búsquedas, recorridos, navegación y ordenamiento.

Flujo del método:

14. Se crean carpetas de primer nivel como `docs`, `imgs`, `music`, `config` y `projects`.

15. Dentro de cada carpeta se agregan subcarpetas y archivos.

16. Se alterna entre `cd()`, `mkdir()`, `touch()`, `back()` y `goToRoot()` para posicionarse correctamente en cada sección.

17. Al final, el árbol queda organizado con varias ramas y diferentes tipos de contenido.

Propósito: generar una estructura compleja para probar el comportamiento del explorador sin tener que crear manualmente cada elemento en tiempo de ejecución.

goToRoot()

Este método devuelve la ubicación actual a la raíz del sistema de archivos.

Flujo del método:

18. Se asigna `root` a la variable `current`.

19. La navegación queda posicionada en el nodo principal.

Propósito: facilitar el regreso inmediato al inicio del árbol.

pwd()

Este método devuelve la ruta completa de la ubicación actual.

Flujo del método:

20. Llama a la versión privada `pwd(current)`.

21. Esa versión construye la ruta recorriendo los padres del nodo actual.

22. Finalmente, retorna la ruta completa en formato `string`.

Propósito: representar la ubicación actual dentro del árbol de forma similar a una terminal.

back()

Este método retrocede a la ubicación anterior registrada en el historial.

Flujo del método:

23. Verifica si la pila back está vacía.
24. Si está vacía, retorna false porque no hay más rutas anteriores.
25. Si existe historial, empuja la ubicación actual a la pila forward.
26. Extrae el último nodo guardado en back.
27. Asigna ese nodo como ubicación actual.
28. Retorna true.

Propósito: permitir navegación reversa sin perder la posición actual.

forward()

Este método avanza a una ubicación que fue abandonada después de usar back().

Flujo del método:

29. Verifica si la pila forward está vacía.
30. Si está vacía, retorna false.
31. Si existe historial, guarda la ubicación actual en la pila back.
32. Extrae el nodo superior de forward.
33. Lo asigna como ubicación actual.
34. Retorna true.

Propósito: restaurar el avance de navegación después de haber retrocedido.

pwd(NodoFS nodo)

Este método privado construye la ruta completa de cualquier nodo recibido.

Flujo del método:

35. Si el nodo es la raíz, retorna /raíz.
36. Se crea una pila de cadenas para invertir el orden de la ruta.
37. Se recorre el nodo actual y luego sus padres hasta llegar a null.
38. Cada nombre se guarda en la pila.
39. Después se extraen los elementos de la pila para armar la ruta desde la raíz hasta el nodo actual.

Propósito: formar rutas correctas tanto para carpetas como para archivos.

searchDFS(String nombre)

Este método busca un nodo usando profundidad primero (DFS).

Flujo del método:

40. Se crea una pila de carpetas y se coloca la raíz.
41. Se inicializa el contador visitedNodes en cero.
42. Mientras la pila no esté vacía, se extrae una carpeta.
43. Se recorre cada hijo de esa carpeta.
44. Por cada hijo visitado, se incrementa el contador.
45. Si el hijo es un archivo, se compara su nombre completo con el parámetro recibido.
46. Si el hijo es una carpeta, se compara su nombre.
47. Cuando se encuentra coincidencia, se retorna la ruta con pwd(child) y la cantidad de nodos visitados.
48. Si no se encuentra, se devuelve el mensaje de no encontrado junto con el total de nodos visitados.

Propósito: localizar un nodo siguiendo un recorrido en profundidad y mostrar cuántos nodos se revisaron durante la búsqueda.

searchBFS(String nombre)

Este método busca un nodo usando amplitud primero (BFS).

Flujo del método:

49. Se crea una cola de carpetas y se coloca la raíz.
50. Se inicializa el contador visitedNodes.
51. Mientras la cola no esté vacía, se extrae la carpeta al frente.
52. Se recorren sus hijos uno por uno.
53. Se incrementa el contador por cada nodo visitado.
54. Si el hijo es un archivo, se compara su nombre completo.
55. Si el hijo es una carpeta, se compara su nombre.
56. Si hay coincidencia, se retorna la ruta y la cantidad de nodos visitados.
57. Si no aparece el nodo, se devuelve el mensaje correspondiente con el conteo total.

Propósito: encontrar un nodo recorriendo primero los niveles más cercanos a la raíz.

queueAdd(String operacion, String ruta)

Este método agrega una tarea a la cola interna de procesamiento.

Flujo del método:

58. Une la operación y la ruta en una sola cadena.
59. Inserta esa cadena en taskQueue.

Propósito: guardar tareas pendientes para ejecutarlas después en orden FIFO.

queueRun()

Este método procesa todas las tareas almacenadas en la cola.

Flujo del método:

60. Verifica si la cola está vacía.
61. Si no hay tareas, retorna queue: no tasks to process.
62. Si hay elementos, crea un acumulador de resultados.
63. Extrae cada tarea de la cola una por una.
64. Por cada tarea, agrega el texto Processing: ... done.
65. Devuelve el resumen de todo lo ejecutado.

Propósito: simular la ejecución secuencial de operaciones pendientes.

3. MÉTODOS AGREGADOS EN Clase LinkedListGeneric

sort(Comparator<T> comparator)

Este método ordena la lista completa usando el comparador recibido.

Flujo del método:

66. Llama a mergeSort(head, comparator).
67. Reemplaza head con la nueva cabeza ya ordenada.

Propósito: ofrecer un ordenamiento flexible para cualquier tipo de dato almacenado en la lista.

mergeSort(ListNode<T> node, Comparator<T> comparator)

Este método divide la lista en dos partes, las ordena de forma recursiva y luego las une.

Flujo del método:

68. Si el nodo es null o tiene un solo elemento, se retorna de inmediato porque ya está ordenado.
69. Se recorre la lista para contar sus elementos.
70. Se calcula la mitad.

71. Se avanza hasta el nodo anterior al punto medio.
72. Se separa la lista en dos mitades.
73. Se llama recursivamente a mergeSort para cada mitad.
74. Se combinan ambas partes con merge().

Propósito: aplicar el algoritmo merge sort sobre la lista enlazada.

merge(ListNode<T> left, ListNode<T> right, Comparator<T> comparator)

Este método fusiona dos sublistas ya ordenadas en una sola lista ordenada.

Flujo del método:

75. Se crea un nodo auxiliar para construir el resultado.
76. Mientras ambas listas tengan elementos, se comparan sus datos.
77. Se enlaza primero el elemento menor o igual, según el comparador.
78. Cuando una lista se agota, se agregan los elementos restantes de la otra.
79. Se retorna la lista resultante.

Propósito: unir dos secuencias ordenadas sin perder el orden final.

4. MÉTODOS AGREGADOS EN Clase Carpeta

listChildrenLong()

Este método muestra el contenido de la carpeta con formato largo.

Flujo del método:

80. Recorre la lista de hijos.
81. Si el hijo es una carpeta, imprime su fecha de creación y su nombre con / al final.
82. Si el hijo es un archivo, imprime su fecha de creación y su representación textual completa.
83. Acumula todo en un StringBuilder y lo devuelve.

Propósito: presentar información detallada de los elementos contenidos en la carpeta.

sortChildBySize()

Este método ordena los hijos de la carpeta por tamaño de menor a mayor.

Flujo del método:

84. Llama al método sort() de la lista enlazada.
85. Usa un comparador que obtiene el tamaño del archivo si el nodo es un Archivo.
86. Si el nodo es una carpeta, se asigna tamaño 0.
87. Se comparan los tamaños y la lista queda ordenada.

Propósito: organizar el contenido de la carpeta según el tamaño de los archivos y la posición de las carpetas.

5. NUEVA Clase Main

La clase Main es el punto de entrada del programa.

main(String[] args)

Flujo del método:

88. Se ejecuta FileExplorer.initFileExplorer().
89. Con eso inicia la aplicación completa.

Propósito: arrancar el sistema desde una sola llamada centralizada.

6. Resumen general del flujo

El programa inicia en Main, luego se carga el explorador principal. Desde FileExplorer, los comandos del usuario se interpretan y se envían a FileSystem. Allí se realizan operaciones sobre el árbol, como navegación, búsqueda, listado y cola de tareas. En paralelo, Carpeta administra el contenido interno de cada carpeta y LinkedListGeneric aporta el ordenamiento de los elementos.

Este conjunto de clases permite que el sistema funcione como un explorador de archivos estructurado, con navegación tipo terminal, búsquedas por DFS y BFS, historial de movimiento y organización de contenido.

Análisis de Complejidad Práctico

3. Escenario de prueba y medición de búsquedas

El objetivo de este escenario es medir, de forma empírica, cuántos nodos recorre el sistema al buscar elementos dentro de un árbol de archivos. Se comparan dos estrategias:

- **DFS (Depth-First Search):** recorre primero una rama hasta el fondo y luego retrocede.
- **BFS (Breadth-First Search):** recorre el árbol por niveles, de arriba hacia abajo.

La métrica usada en este informe es la siguiente:

Nodo recorrido = cada nodo que el algoritmo extrae de la pila o cola, lo compara con el nombre buscado y decide si continúa o termina.

Con esta definición, la tabla de resultados refleja el comportamiento real de la búsqueda y no una estimación teórica.

3.2 Árbol de prueba

El árbol fue construido con los comandos del proyecto y contiene **33 nodos en total: 14 carpetas y 19 archivos**. La profundidad máxima es de **4 niveles** y el árbol mezcla carpetas y archivos en distintas ramas, cumpliendo el requisito de tener nodos a diferentes profundidades.

```
raiz/                                     <- nivel 0
├── docs/                                 <- nivel 1
│   ├── work/                             <- nivel 2
│   │   ├── informe.pdf                   (100KB)    <- nivel 3
│   │   ├── presentacion.pptx            (80KB)     <- nivel 3
│   │   └── presupuesto.xlsx              (60KB)     <- nivel 3
│   ├── university/                       <- nivel 2
│   │   ├── apuntes.txt                   (200KB)    <- nivel 3
│   │   └── tesis.pdf                     (300KB)    <- nivel 3
│   └── cv.pdf                             (120KB)    <- nivel 2
├── imgs/                                 <- nivel 1
│   ├── vacaciones/                       <- nivel 2
│   │   ├── playa.jpg                    (33KB)     <- nivel 3
│   │   └── montana.jpg                   (40KB)     <- nivel 3
│   ├── perfil.jpg                        (40KB)     <- nivel 2
│   └── logo.png                          (65KB)     <- nivel 2
└── music/                                <- nivel 1
    ├── rap/                              <- nivel 2
    └── AllEyezOnMe.wav                    (1000KB)   <- nivel 3
```

```

├── 21Questions.mp3      (300KB)      <- nivel 3
├── reggae/              <- nivel 2
│   ├── ThreeLittleBirds.wav (2000KB) <- nivel 3
│   └── Jamming.mp3        (500KB)    <- nivel 3
├── config/              <- nivel 1
│   └── settings.json      (5KB)      <- nivel 2
├── projects/            <- nivel 1
│   ├── fileexplorer/     <- nivel 2
│   │   ├── Main.java      (100KB)    <- nivel 3
│   │   └── README.md      (30KB)    <- nivel 3
│   ├── stockflow/        <- nivel 2
│   │   ├── backend/       <- nivel 3
│   │   │   └── SaleRepository.java (120KB) <- nivel 4
│   │   └── frontend/      <- nivel 3
│   │       └── App.tsx      (110KB)    <- nivel 4

```

Resumen del árbol

Métrica	Valor
Total de nodos	33
Carpetas	14
Archivos	19
Profundidad máxima	4
Ancho máximo	15 nodos

3.3 Resultados de búsqueda medidos

Los siguientes resultados fueron obtenidos con el comando search del proyecto. La comparación se hizo sobre los mismos nodos y con la misma estructura de árbol.

#	Nodo buscado	Niv el	DFS: nodos visitados	BFS: nodos visitados	Observación
1	App.tsx	4	10	33	DFS llega antes a la rama projects/stockflow/frontend
2	AllEyezOnMe.wav	3	19	24	DFS encuentra antes la rama music/rap
3	SaleRepository.java	4	11	32	DFS baja rápidamente por projects/stockflow/backend
4	noexiste.txt	—	33	33	Ambos recorren todo el árbol
5	settings.json	2	14	14	Ambos lo encuentran en la misma posición
6	informe.pdf	3	31	17	BFS lo encuentra antes por recorrido por niveles

3.4 Interpretación técnica

DFS

DFS es más eficiente cuando el nodo buscado está en una rama que se profundiza temprano. En este árbol ocurrió con:

- `App.tsx` → 10 nodos visitados
- `SaleRepository.java` → 11 nodos visitados
- `AllEyezOnMe.wav` → 19 nodos visitados

Esto sucede porque DFS recorre una rama completa antes de pasar a la siguiente, por lo que puede llegar muy rápido a nodos profundos si la rama correcta se explora pronto.

BFS

BFS es más eficiente cuando el nodo buscado está cerca de la raíz o en un nivel intermedio que se visita temprano por orden de niveles. En este árbol ocurrió con:

- `informe.pdf` → 17 nodos visitados
- `AllEyezOnMe.wav` → 24 nodos visitados
- `App.tsx` → 33 nodos visitados

Esto se debe a que BFS primero agota el nivel 1, luego el nivel 2, después el nivel 3 y así sucesivamente.

Caso de peor escenario

Cuando el nodo no existe (`noexiste.txt`), ambos algoritmos deben revisar todos los nodos del árbol. Por eso ambos reportan **33 nodos visitados**. Este resultado confirma que la complejidad en el peor caso es **$O(N)$** para DFS y BFS.

Caso con empate

`settings.json` aparece con **14 nodos visitados** en ambos recorridos. Esto no significa que DFS y BFS sean iguales en general, sino que en esta estructura concreta el nodo queda en una posición equivalente para ambas estrategias.